SciPy in ten minutes

What it is, and the two pieces this project uses

NumPy holds the numbers. SciPy does things with them.

NumPy gives you arrays and fast arithmetic on them. SciPy is a library of ready-made numerical algorithms that take NumPy arrays in and hand NumPy arrays back: optimisation, geometry, integration, statistics, signal and image processing. You import the one submodule you need. There is nothing to set up and no new data type to learn. The habit to build is: before writing a numerical routine yourself, check whether SciPy already has a tested one.

1. scipy.optimize.minimize: find the lowest point of a function

You give it a function of a vector, a starting guess, and ideally the gradient (the vector of slopes). It walks downhill and returns the lowest point it finds. With jac=True your function returns the value and the gradient together.

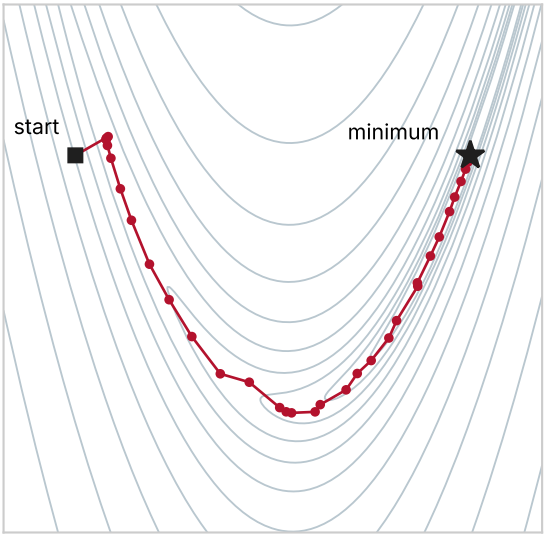
```
from scipy.optimize import minimize

def f(v):                        # value, gradient
    x, y = v
    value = (1-x)**2 + 100*(y-x*x)**2
    grad  = [-2*(1-x) - 400*x*(y-x*x),
             200*(y-x*x)]
    return value, grad

res = minimize(f, x0=[-1.2, 1.0],
               jac=True, method="L-BFGS-B")
res.x        # -> [1.0, 1.0]
```

The method "L-BFGS-B" is a good default when you have a gradient: it learns the curvature of the function from the last few steps, so it needs far fewer steps than plain gradient descent.

The path minimize took: 36 steps along a curved valley. Grey lines join points of equal height.



2. scipy.spatial.Voronoi: who is nearest?

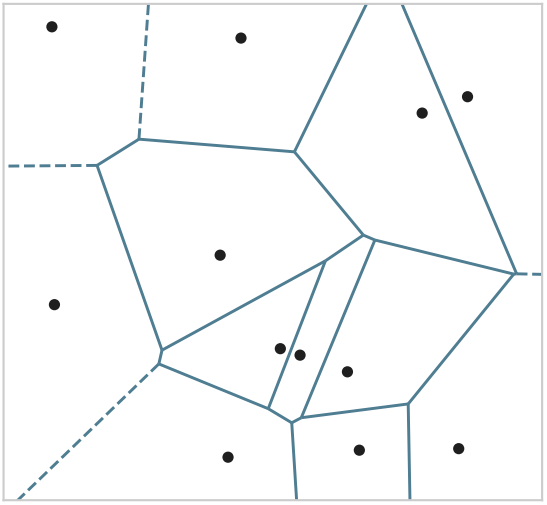
Give it points ("sites"). It returns the map in which every location belongs to its nearest site. Each site's territory is a convex polygon called its cell.

```
from scipy.spatial import Voronoi

sites = rng.uniform(0, 1, size=(12, 2))
vor = Voronoi(sites)
vor.vertices      # corner coordinates
vor.regions       # corners of each cell
vor.point_region  # which cell is site i's
```

Cells on the outside run off to infinity; SciPy marks those with a -1 in the corner list. This project needs cells of chosen sizes, which plain Voronoi cannot give, so we use SciPy's version only as an independent check on our own code (page 2).

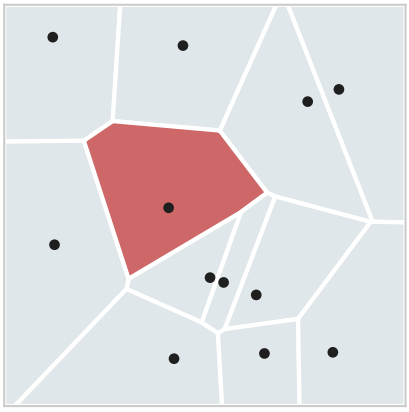
Twelve sites and their Voronoi cells. Dashed edges continue to infinity.



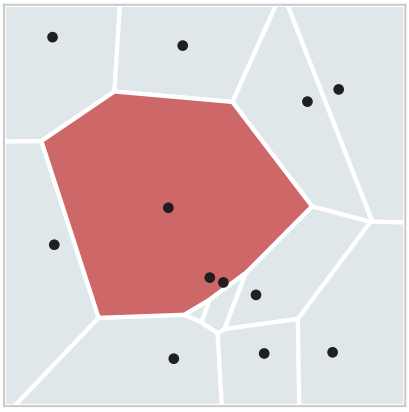
3. How this project uses SciPy

The chart needs each donor's cell to have an area proportional to the money given. The trick is to give every site a weight. A point x belongs to the site with the smallest value of (distance to the site) squared minus (the site's weight). Raise a weight and that cell grows at its neighbours' expense; the edges stay straight and the cells stay convex. This weighted version is called a power diagram.

All weights equal: the ordinary Voronoi diagram.



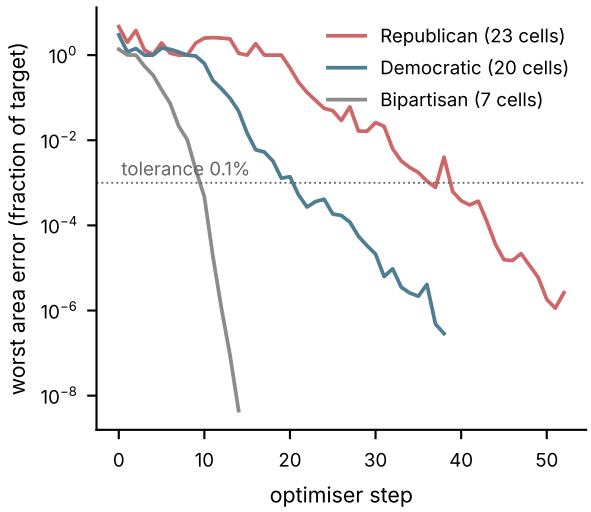
One weight raised: that cell grows, the rest give way.



So the job is: find the weights that give every cell its target area. That is a "find the lowest point" problem, and minimize solves it. Our function's gradient is simply (cell area minus target area), so the lowest point is exactly where every area is right. The function is bowl-shaped (convex), which means there is one answer and downhill always leads to it.

```
def objective(w, sites, targets, boundary):
    cells = power_cells(sites, w, boundary)
    areas = [polygon_area(c) for c in cells]
    return value, areas - targets   # gradient
res = minimize(objective, w0, jac=True,
               args=(sites, targets, boundary),
               method="L-BFGS-B")
```

Real donor data, random starting sites: worst error in any cell's area, step by step.



4. Reading what minimize returns

```
res.x        # the answer: here, one weight per cell      res.fun      # the function's value there
res.success  # True if a stopping test was met           res.message  # why it stopped, in words
res.nit      # number of steps taken
```

res.success only says the optimiser's own stopping rule fired. This project measures what it cares about (the worst area error) after every solve and raises an error above 0.1%.

5. Three mistakes worth knowing in advance

It only minimises. To maximise a function F, hand it -F and also negate the gradient. Our F is concave (dome-shaped) and we want its top, so objective() returns -F and (areas - targets).

A wrong gradient fails quietly. If the sign or an index is off, minimize still runs and returns something, slowly or wrongly. scipy.optimize.check_grad compares your gradient with a numerical estimate. The tests do this, and also check that with equal weights our cells match scipy.spatial.Voronoi corner for corner.

An impossible problem does not announce itself. Cells always fill the region, so target areas must add up to its area. Building this page, targets summing to 1 were passed for a disc of area 0.9998: minimize ran, reported nothing wrong, and sent the weights to 10^23. The code now refuses that.